승객 100번이 2025년 1월 1일 이후 탑승한 데이터

```
SELECT p.pid, p.pname, f.fid, f.src, f.dest, b.fdate
FROM Passenger p
JOIN Booking b ON p.pid = b.pid
JOIN Flight f ON b.fid = f.fid
WHERE p.pid = 100
AND b.fdate > TO_DATE('2025-01-01', 'YYYY-MM-DD');
```

작성 순서	실행 순서
SELECT	FROM
FROM	JOIN
JOIN	WHERE
WHERE	SELECT

① FROM Passenger p
테이블을 먼저 가져옴 — Passenger 전체 행 로드

↓

② JOIN Booking b ON p.pid = b.pid
Passenger ↔ Booking 결합 — pid가 같은 행끼리 연결

↓

③ JOIN Flight f ON b.fid = f.fid
(Passenger+Booking) ↔ Flight 결합 — fid가 같은 행끼리 연결

↓

④ WHERE p.pid = 100 AND b.fdate > TO_DATE(...)
결합된 전체 행 중 조건에 맞는 행만 필터링
조건 1: pid = 100 / 조건 2: 예약일이 2025-01-01 이후

↓

⑤ SELECT p.pid, p.pname, f.fid, f.src, f.dest, b.fdate
필터된 행에서 원하는 컬럼만 추출
alias(p, b, f)는 여기서 비로소 컬럼 참조에 사용됨

↓

⑥ 결과 반환
pid=100 승객의 2025-01-01 이후 예약 항공편 목록

항공편 100번인 승객이 2025년 1월 1일 이후 탑승한 데이터

```
SELECT p.pid, p.pname, f.fid, f.src, f.dest, b.fdate
FROM Passenger p
JOIN Booking b ON p.pid = b.pid
JOIN Flight f ON b.fid = f.fid
WHERE f.fid = 100
AND b.fdate > TO_DATE('2025-01-01', 'YYYY-MM-DD');
```

작성 순서	실행 순서
SELECT	FROM
FROM	JOIN
JOIN	WHERE
WHERE	SELECT

① FROM Passenger p
Passenger 테이블 전체를 읽어옴
별칭 p 등록 — 모든 승객 행이 대상

↓

② JOIN Booking b ON p.pid = b.pid
Passenger 각 행 ↔ Booking 행을 pid 기준으로 결합
결과: (p.pid, p.pname, b.pid, b.fid, b.fdate …) 중간 테이블 생성
한 승객이 예약이 여러 개면 → 여러 행으로 펼쳐짐

↓

③ JOIN Flight f ON b.fid = f.fid
(Passenger+Booking) 중간 테이블 ↔ Flight를 fid 기준으로 결합
결과: p.pid, p.pname, b.fdate, f.fid, f.src, f.dest … 모두 한 행에 존재
이 시점에서 세 테이블 컬럼이 전부 합쳐진 넓은 임시 테이블 완성

↓

④ WHERE f.fid = 100 AND b.fdate > TO_DATE('2025-01-01','YYYY-MM-DD')
임시 테이블 전체 행을 두 조건으로 동시에 필터링 (AND → 둘 다 충족해야 통과)

조건 1: f.fid = 100
조건 2: b.fdate > 2025-01-01
→ fid=100 항공편을, 2025년 1월 1일 이후에 예약한 승객 행만 남음

↓

⑤ SELECT p.pid, p.pname, f.fid, f.src, f.dest, b.fdate
WHERE를 통과한 행들에서 6개 컬럼만 추출
p.pid · p.pname — 승객 정보
f.fid · f.src · f.dest — 항공편 정보 (fid=100 고정) · b.fdate — 예약일

↓

⑥ 결과 반환
fid=100 항공편을 2025-01-01 이후에 예약한 승객 목 ↓ 러 명 가능)

예약을 한 적이 없는 고객의 이름(pname)을 보이시오

```
SELECT DISTINCT p.pname
FROM Passenger p
WHERE NOT EXISTS (
SELECT 1
FROM Booking b
WHERE b.pid = p.pid );
```

상관커리 실행단계

```
SELECT name FROM Customer c
WHERE EXISTS (
SELECT * FROM Orders o
WHERE o.custid = c.custid
```

← 이전 1 / 4단계 다음 → custid=1 → Orders에 주문 2건 발견! → EXISTS = TRUE → 반환

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	?
박민준	?
정수현	?

← 이전 2 / 4단계 다음 → custid=2 → Orders에 주문 1건 발견! → EXISTS = TRUE → 반환

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	EXISTS TRUE → 반환
박민준	?
정수현	?

상관커리 실행단계

```
SELECT name FROM Customer c
WHERE EXISTS (
SELECT * FROM Orders o
WHERE o.custid = c.custid
```

← 이전 3 / 4단계 다음 → custid=3 → Orders에 주문 1건 발견! → EXISTS = TRUE → 반환

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	EXISTS TRUE → 반환
박민준	EXISTS TRUE → 반환
정수현	?

← 이전 4 / 4단계 다음 → custid=4 → Orders에 주문 없음 → EXISTS = FALSE → 제외

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	EXISTS TRUE → 반환
박민준	EXISTS TRUE → 반환
정수현	EXISTS FALSE → 제외